

Beyond Trust: Building Community-Driven Security Analysis for Your .NET Software Supply Chain

Niels Tanis - Security Researcher & Engineer



THE IT CONFERENCE FOR
.NET & MICROSOFT TECHNOLOGIES

TIDALIS



Who am I?



- Niels Tanis
- Security Researcher & Engineer
 - Background .NET Development, Pentesting/ethical hacking, and software security consultancy
 - Decade of research on static analysis for .NET apps
 - Software Security Engineer at Tidalis
- Microsoft MVP – Developer Technologies

TIDALIS



TIDALIS



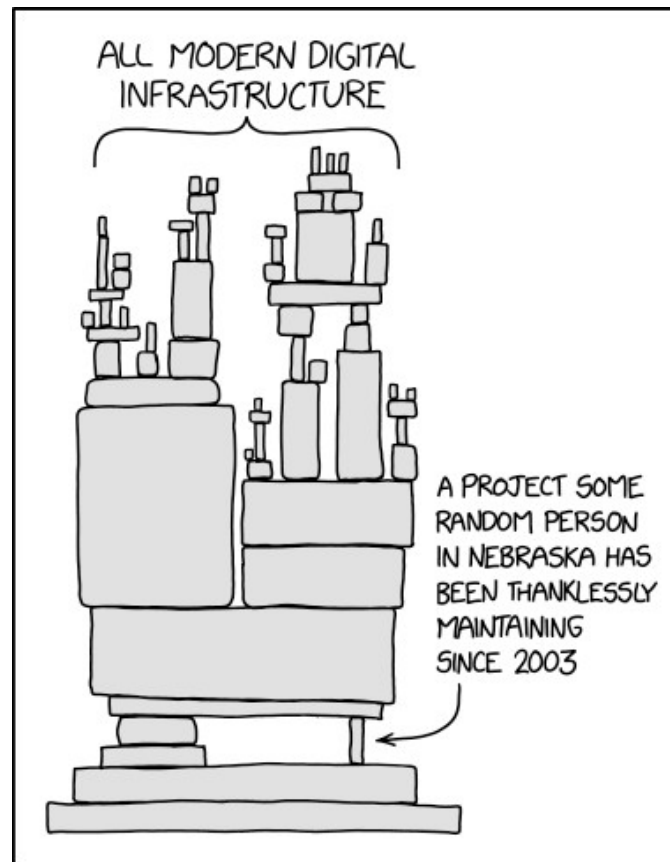
@niels.fennec.dev



@nielstanis@infosec.exchange

Modern Application Architecture

XKCD 2347



TIDALIS

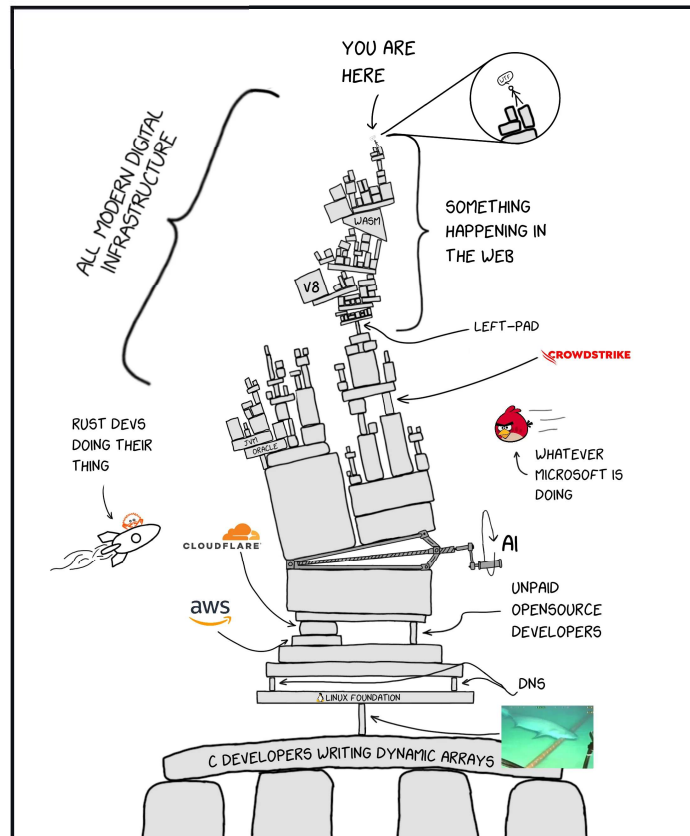


@niels.fennec.dev



@nielstanis@infosec.exchange

You are here



TIDALIS



@niels.fennec.dev



@nielstanis@infosec.exchange

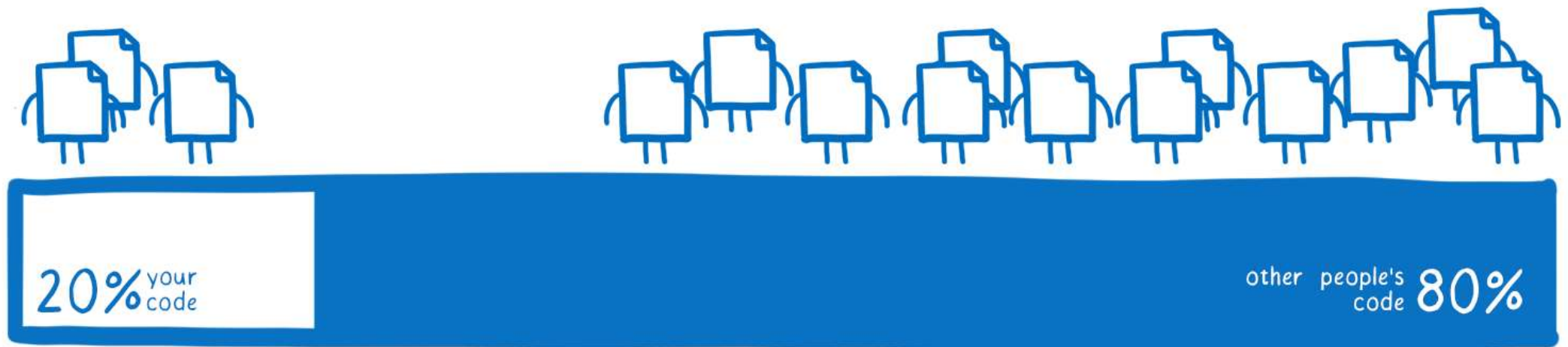
Agenda



- Security challenges in our .NET software supply chain
- How can we start securing our supply-chain?
- Fennec Labs
 - Understanding what projects do for security
 - Know what's inside package
 - Review package
 - Future and idea's
- Conclusion and Q&A



Average codebase composition

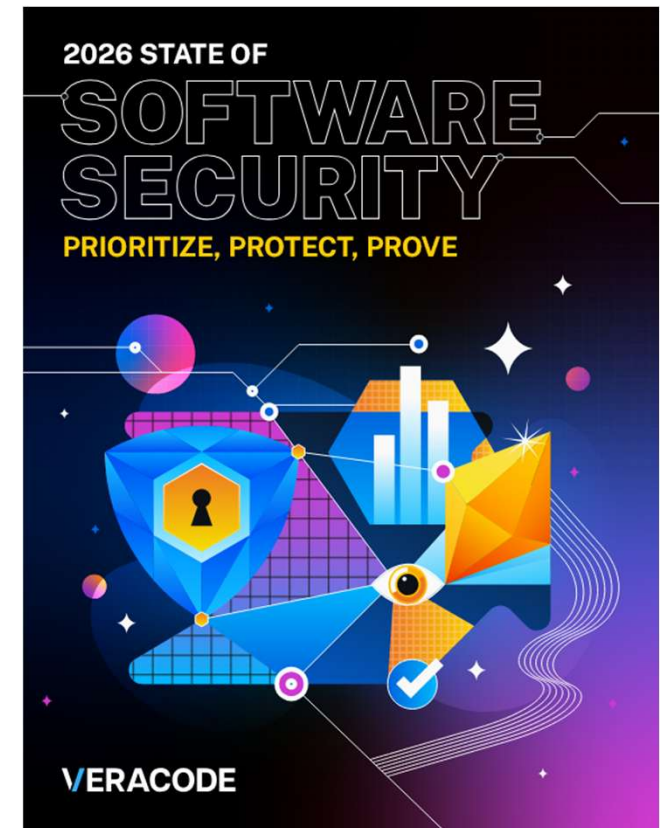
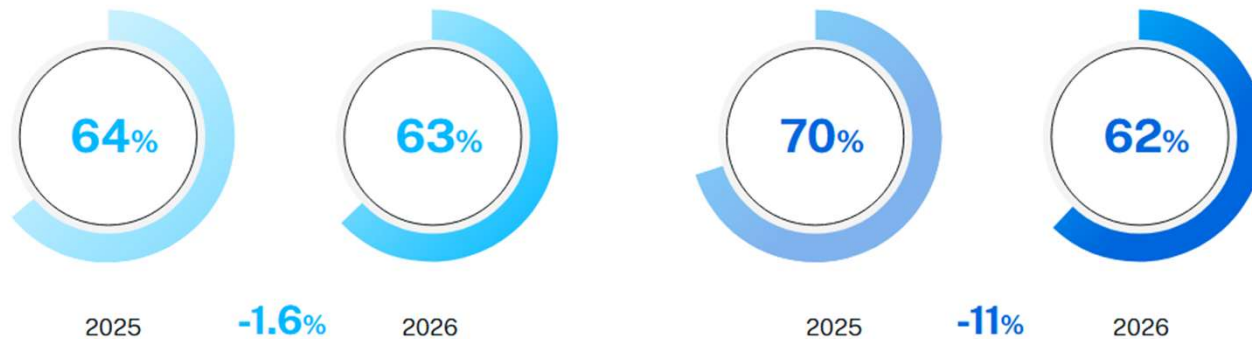


Veracode State Of Software Security 2026



Prevalence of flaws
in first-party (left)
vs. third-party
(right) code among
applications

Percentage of flaws



TIDALIS

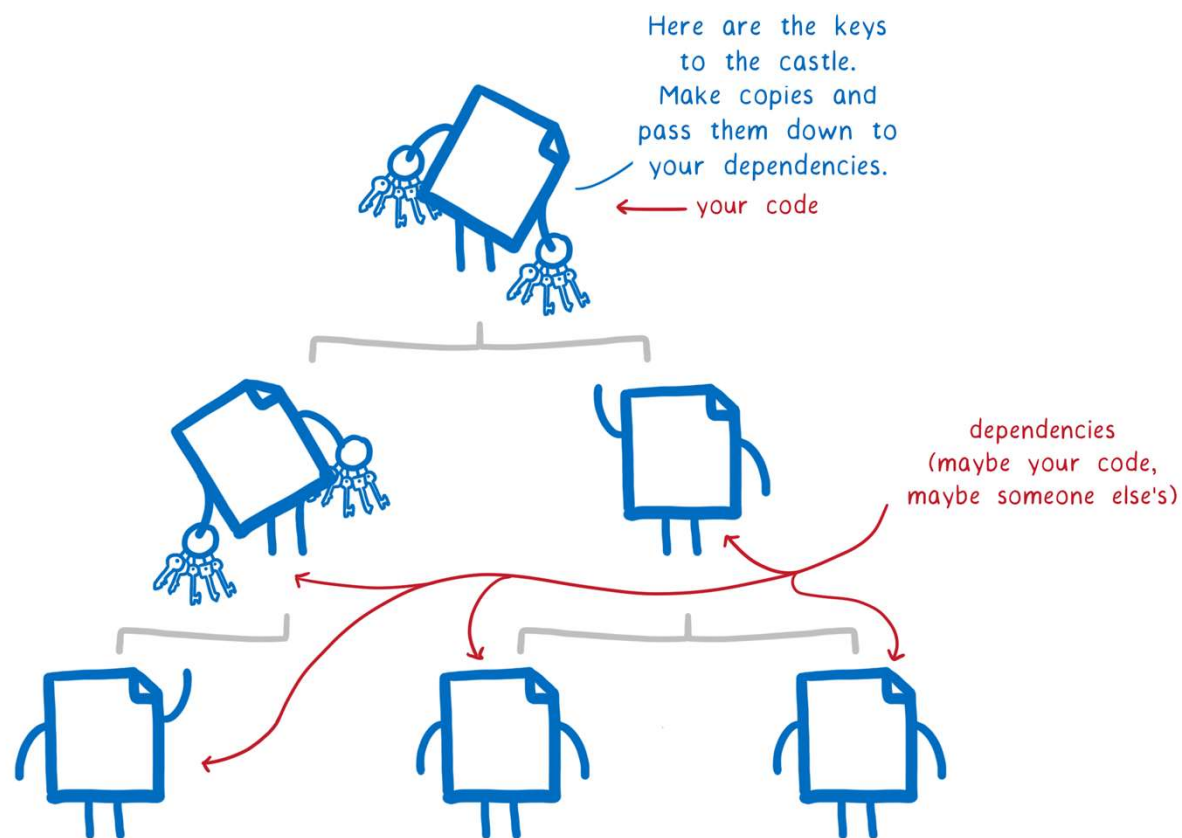


@niels.fennec.dev

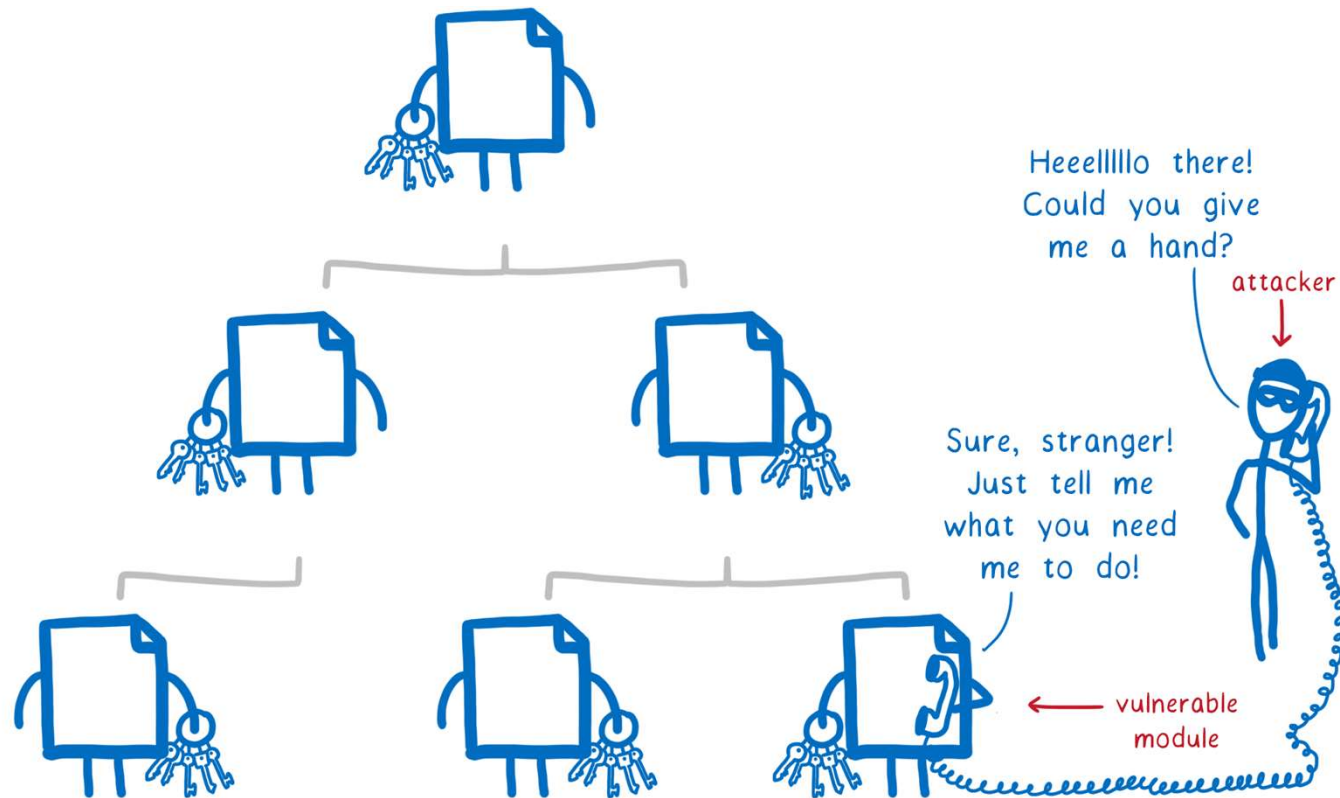


@nielstanis@infosec.exchange

Average codebase composition



Vulnerable Assembly



DotNet CLI



```
nelson@ghost-m2:~/research/consoleapp
└─ nelson at ghost-m2 in ~/research/consoleapp
   └─ dotnet list package
      Project 'consoleapp' has the following package references
         [net9.0]:
         Top-level Package      Requested   Resolved
         > itext7                9.4.0      9.4.0

└─ nelson at ghost-m2 in ~/research/consoleapp
   └─ dotnet list package --vulnerable

The following sources were used:
    https://api.nuget.org/v3/index.json

The given project `consoleapp` has no vulnerable packages given the current sources.
└─ nelson at ghost-m2 in ~/research/consoleapp
   └─
```



DotNet CLI



```
nelson@ghost-m2: ~/research/consoleapp
└─nelson at ghost-m2 in ~/research/consoleapp
  └─dotnet list package --include-transitive
    Project 'consoleapp' has the following package references
    [net9.0]:
    Top-level Package      Requested  Resolved
    > itext7                9.4.0     9.4.0

    Transitive Package                                Resolved
    > itext                                                    9.4.0
    > itext.commons                                             9.4.0
    > Microsoft.CSharp                                         4.0.1
    > Microsoft.DotNet.PlatformAbstractions                   1.1.0
    > Microsoft.Extensions.DependencyInjection                 5.0.0
    > Microsoft.Extensions.DependencyInjection.Abstractions     5.0.0
    > Microsoft.Extensions.DependencyModel                     1.1.0
    > Microsoft.Extensions.Logging                             5.0.0
    > Microsoft.Extensions.Logging.Abstractions                5.0.0
    > Microsoft.Extensions.Options                             5.0.0
    > Microsoft.Extensions.Primitives                          5.0.0
    > Microsoft.NETCore.Platforms                              1.1.1
    > Microsoft.NETCore.Targets                                1.1.3
    > Microsoft.Win32.Primitives                               4.3.0
    > Microsoft.Win32.Registry                                 4.3.0
    > Newtonsoft.Json                                           9.0.1
```



DotNet CLI



```
nelson@ghost-m2:~/research/consoleapp
> dotnet list package --include-transitive --vulnerable

The following sources were used:
  https://api.nuget.org/v3/index.json

Project `consoleapp` has the following vulnerable packages
[net9.0]:
  Transitive Package    Resolved  Severity  Advisory URL
  > Newtonsoft.Json      9.0.1     High      https://github.com/advisories/GHSA-5crp-9r3c-p9vr

nelson at ghost-m2 in ~/research/consoleapp
>
```



DotNet CLI - .NET 10

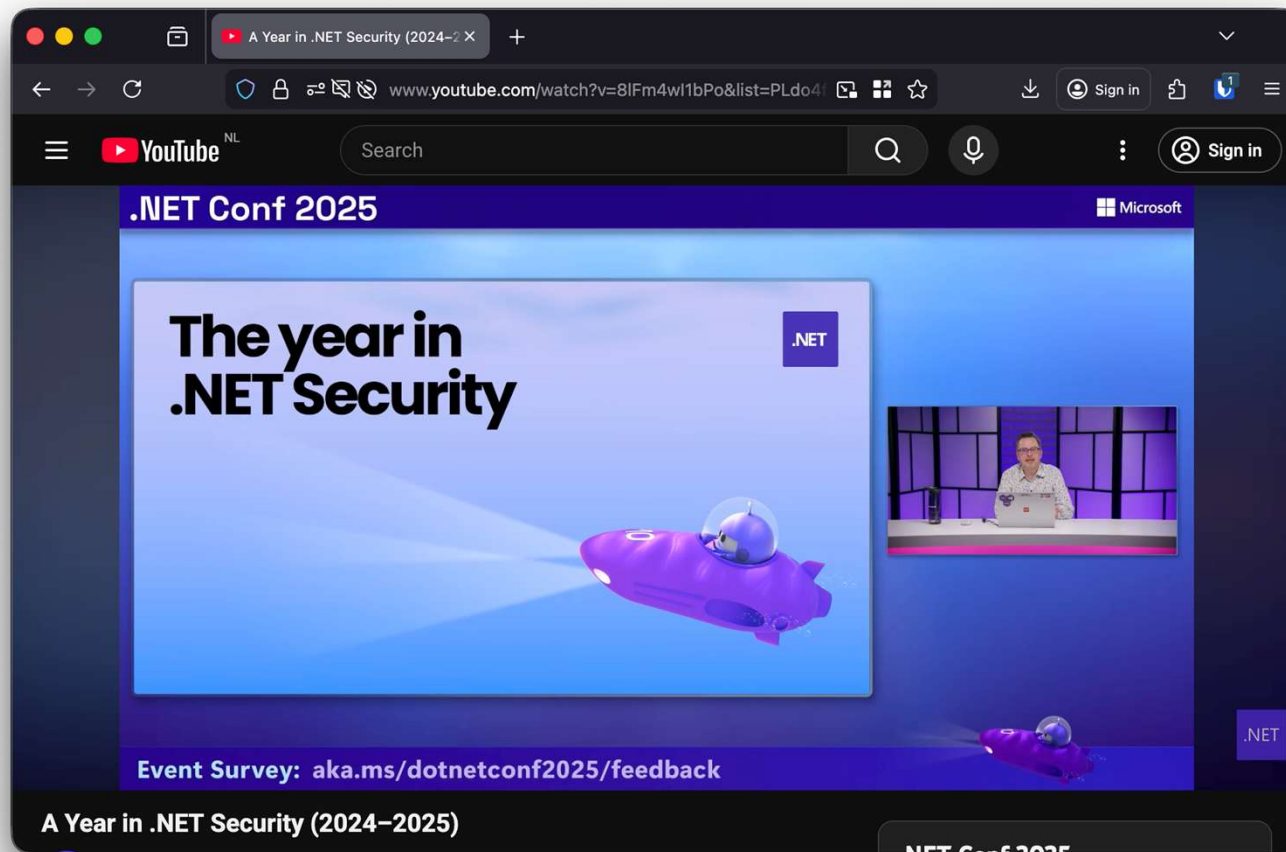


```
nelson@ghost-m2:~/research/consoleapp
> nelson at ghost-m2 in ~/research/consoleapp
└─ dotnet --version
10.0.100
> nelson at ghost-m2 in ~/research/consoleapp
└─ dotnet build
Restore succeeded with 1 warning(s) in 0.3s
  /Users/nelson/Research/consoleapp/consoleapp.csproj : warning NU1903: Package 'Newtonsoft.Json' 9.0.1 has a known high
severity vulnerability, https://github.com/advisories/GHSA-5crp-9r3c-p9vr
  consoleapp net10.0 succeeded with 1 warning(s) (1.5s) → bin/Debug/net10.0/consoleapp.dll
  /Users/nelson/Research/consoleapp/consoleapp.csproj : warning NU1903: Package 'Newtonsoft.Json' 9.0.1 has a known high
severity vulnerability, https://github.com/advisories/GHSA-5crp-9r3c-p9vr

Build succeeded with 2 warning(s) in 2.0s
> nelson at ghost-m2 in ~/research/consoleapp
└─
```



.NET Conf 2025 – The year in .NET Security



TIDALIS



@niels.fennec.dev



@nielstanis@infosec.exchange

.NET Conf 2025 – The year in .NET Security



The screenshot shows a YouTube video player with a presentation slide titled ".NET CVEs 1 of 2". The slide lists six CVEs with their corresponding attack types and descriptions:

01	02	03	04	05	06
CVE-2025-55248	CVE-2025-55247	CVE-2025-30399	CVE-2025-26646	CVE-2025-21173	CVE-2025-21172
Information Disclosure A MITM (man in the middle) attacker may prevent use of TLS between client and SMTP server, forcing client to send data over unencrypted connection.	Denial-of-Service A vulnerability exists in .NET where predictable paths for MSBuild's temporary directories on Linux could lead to DoS of builds.	Remote Code Execution Exploitable by placing files in particular locations, leading to unintended code execution.	Spoofing A vulnerability exists in .NET SDK or MSBuild applications where external control of file name or path allows an unauthorized attacker to perform spoofing over a network.	Elevation of Privilege Exploitable by writing a specially crafted file in the security context of the local system. This only affects .NET on Linux operating systems.	Remote Code Execution Exploitable by loading a specially crafted file in Visual Studio.

Below the slide, the text ".NET Conf Student Zone - Save the date! Nov. 14, 2025" is visible. The video player interface includes a search bar, a "Sign in" button, and a "A Year in .NET Security (2024-2025)" tab.

TIDALIS

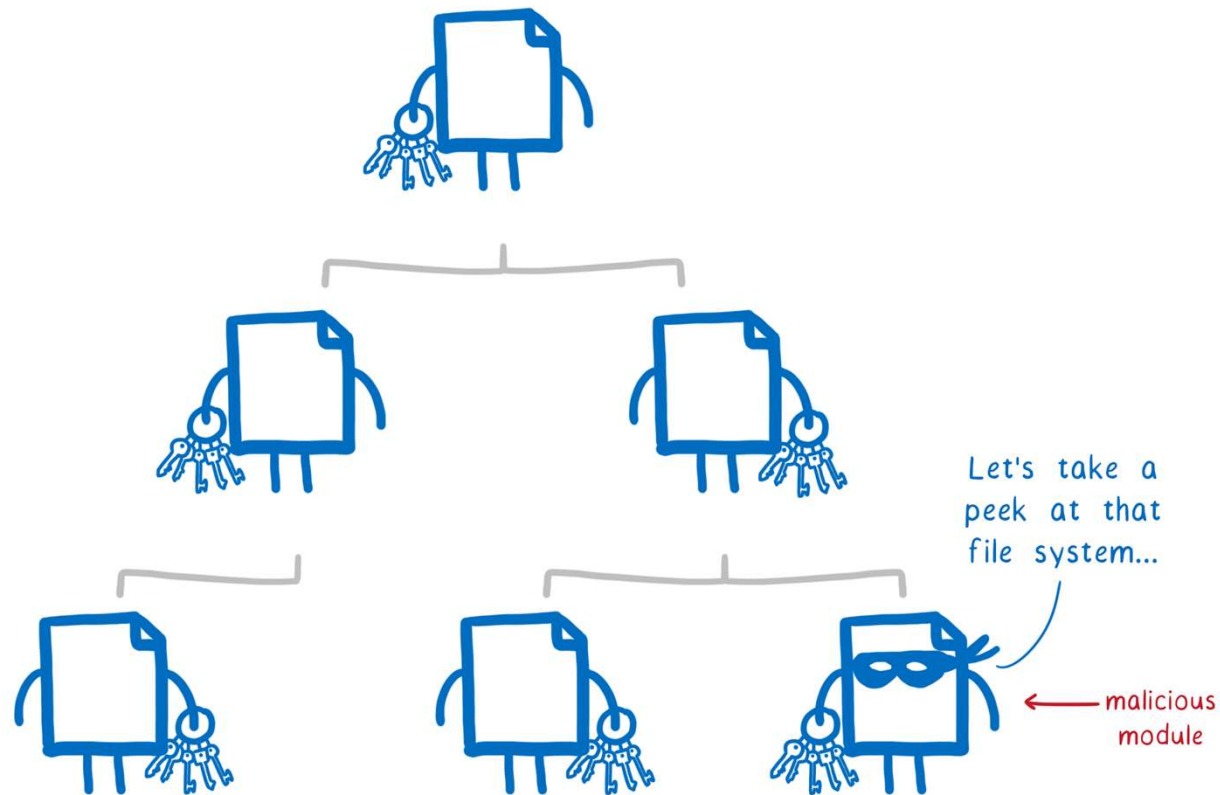


@niels.fennec.dev

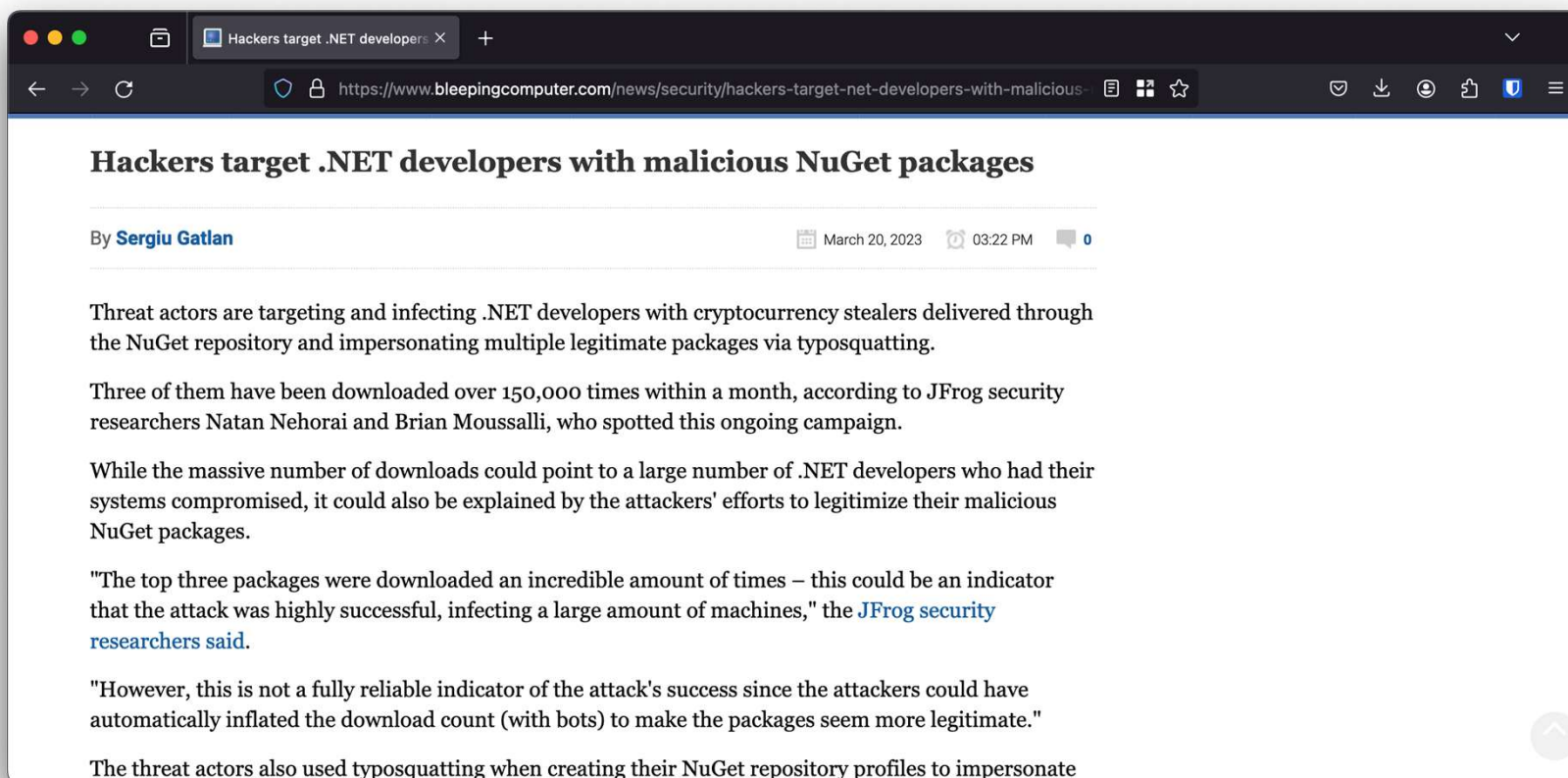


@nielstanis@infosec.exchange

Malicious Assembly



Malicious Package



TIDALIS

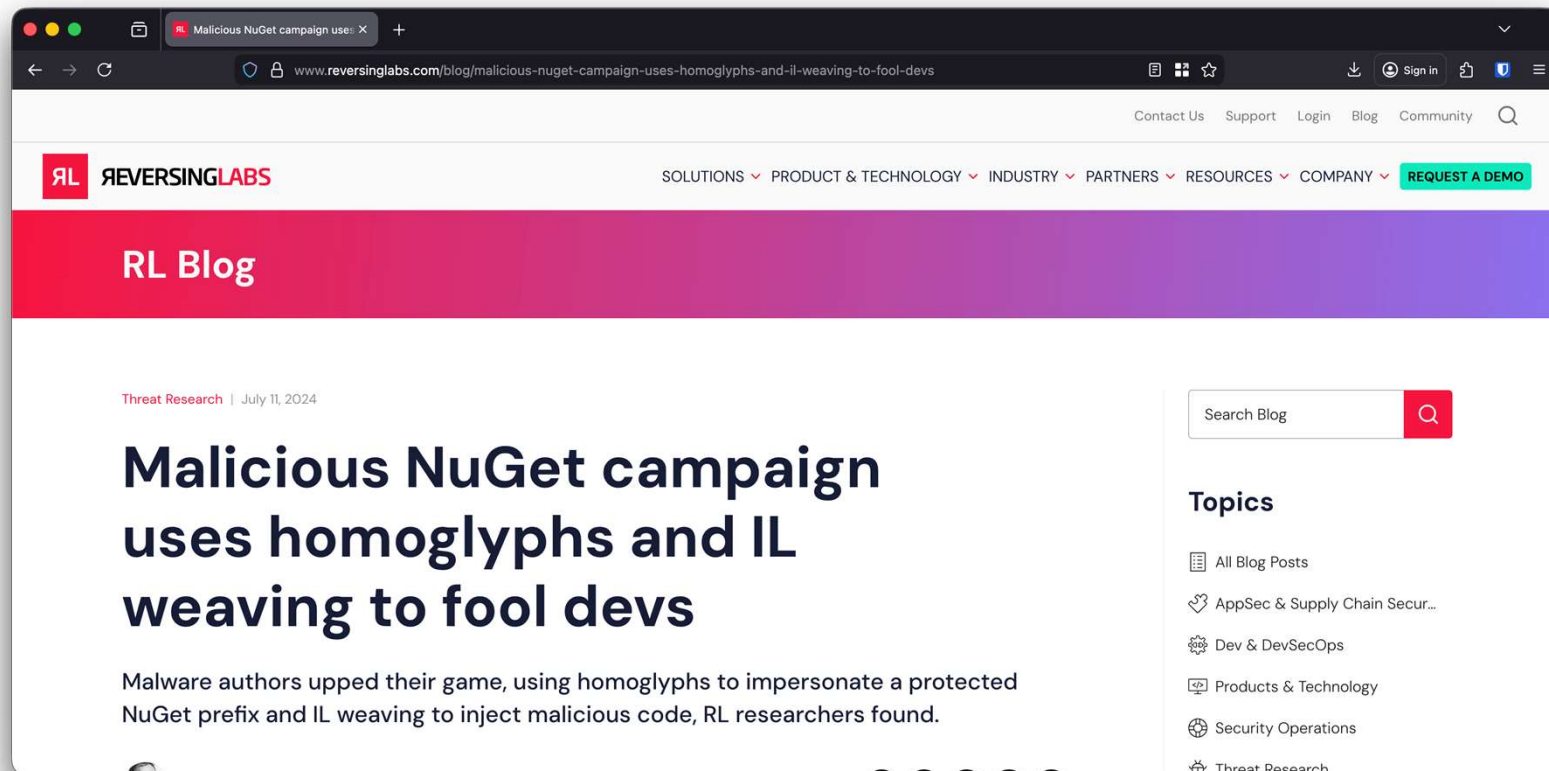


@niels.fennec.dev



@nielstanis@infosec.exchange

Malicious Package



TIDALIS

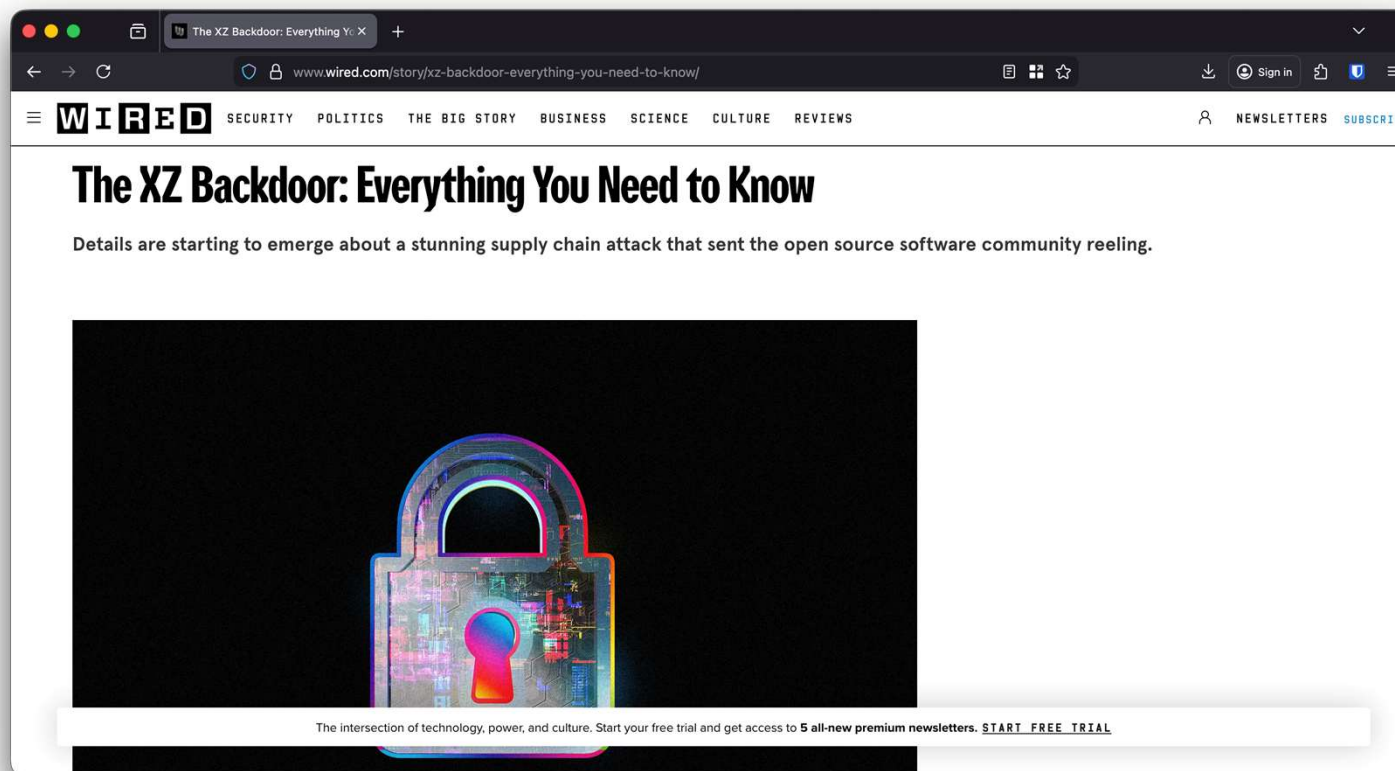


@niels.fennec.dev



@nielstanis@infosec.exchange

Do you know what's inside? – xz utils



TIDALIS

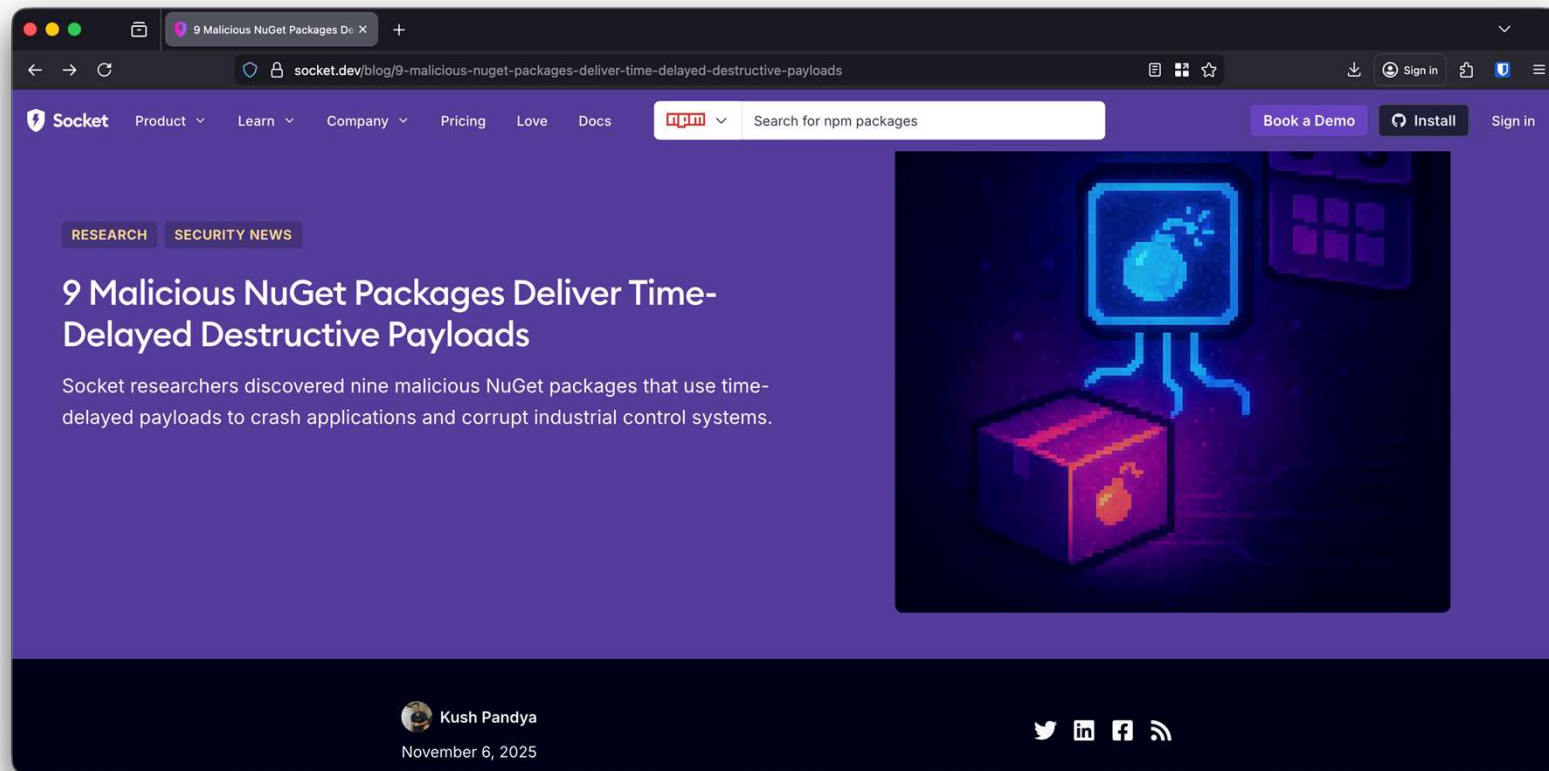


@niels.fennec.dev



@nielstanis@infosec.exchange

Malicious Package



TIDALIS



@niels.fennec.dev



@nielstanis@infosec.exchange

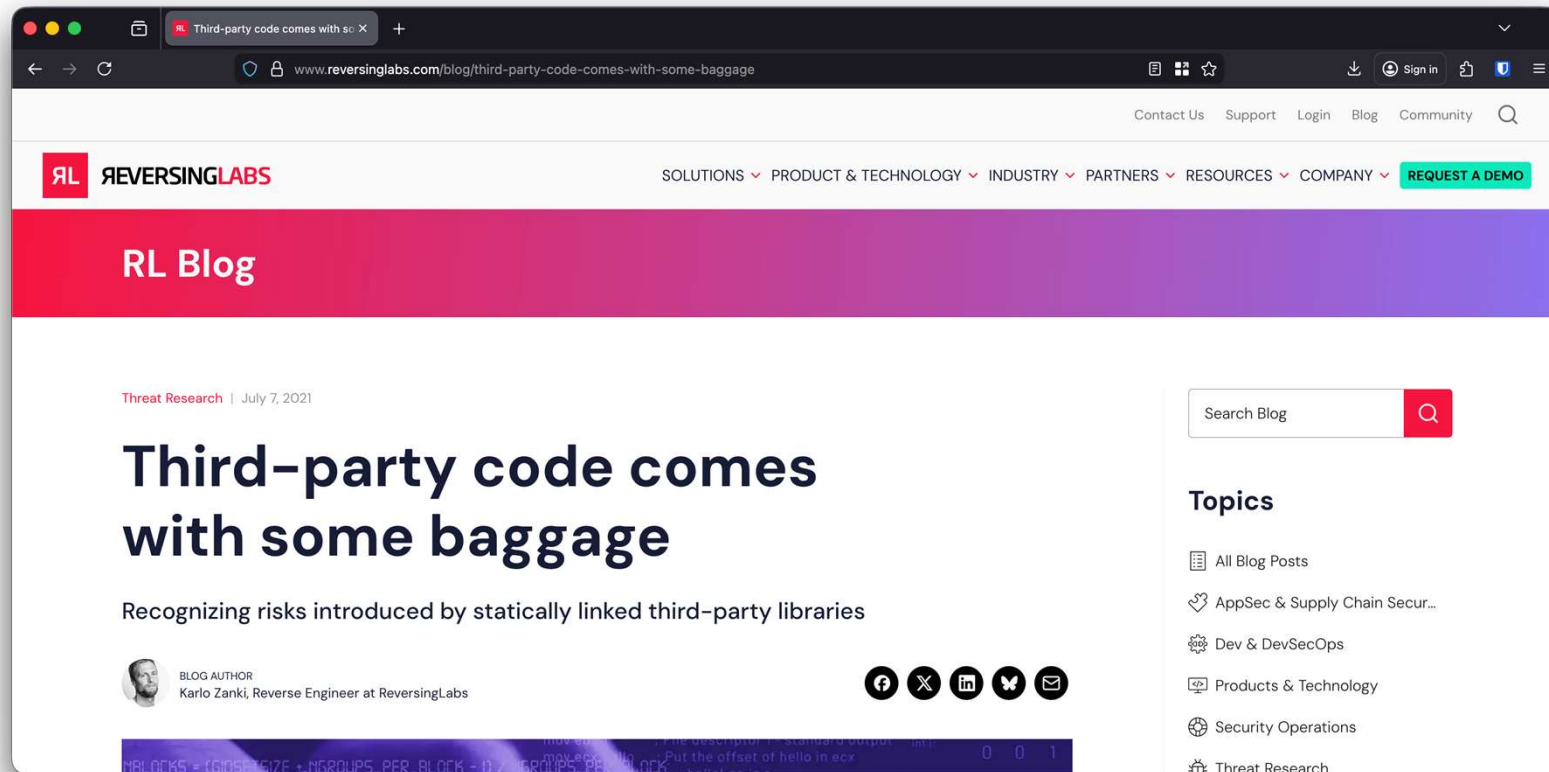
Malicious Package



- Single publisher called **shanghai666**
- 99% code is valid & functional focusing on databases and industrial PLC's, malicious extension methods
- Delayed logic for Aug '27 & Nov '28
- 20% of database queries are terminated
- Sharp7Extend (typo squatting) for PLC, immediate activation mimics hardware failures



Do you know what's inside?



TIDALIS



@niels.fennec.dev



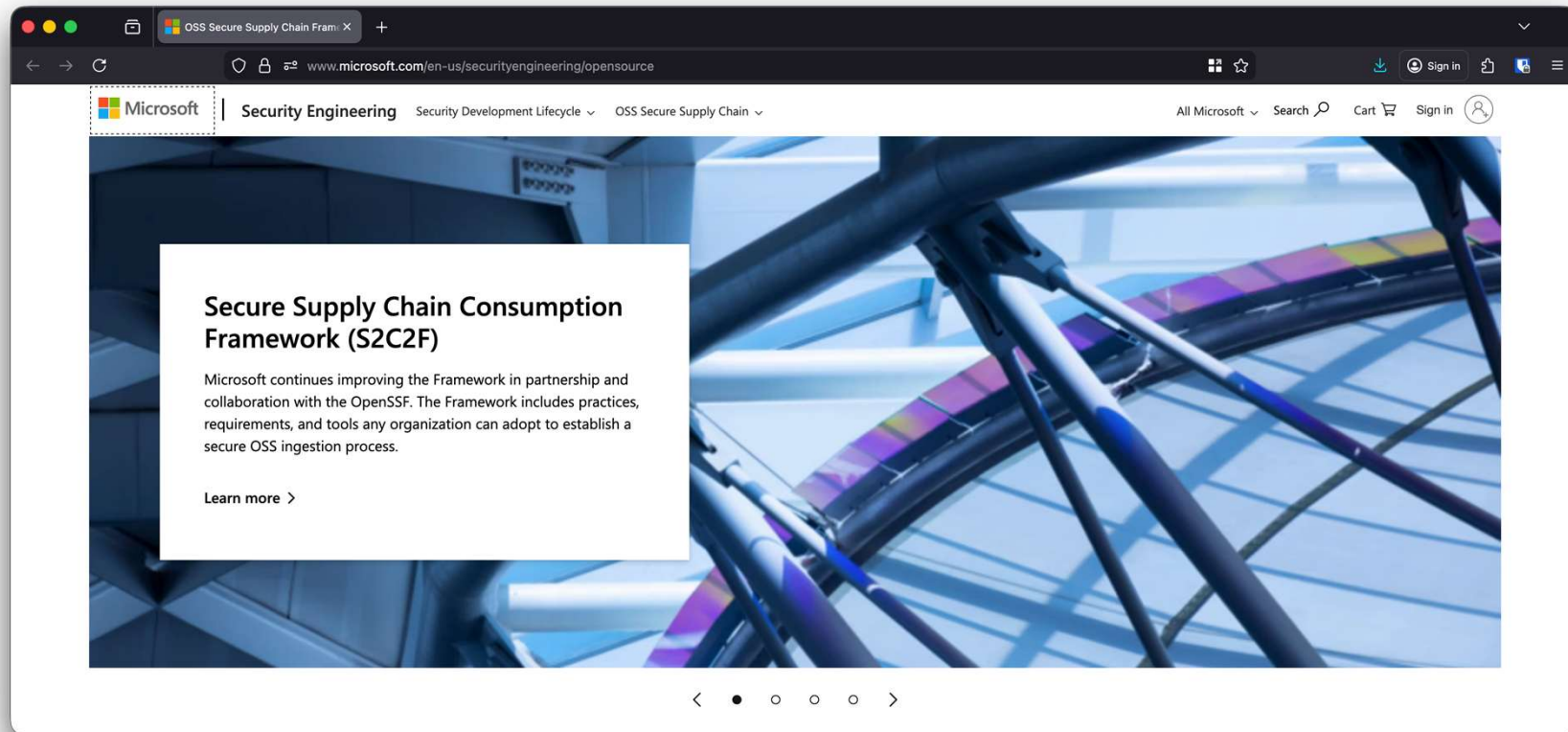
@nielstanis@infosec.exchange

Do you know what's inside?



- Unmanaged components interop
- Precompiled version of 7Zip, WinSCP and PuTTYgen
- Zlib compression library
 - Only source code distributed
 - Used in medical appliances
 - Severe vulnerabilities since 2005!

Secure Supply Chain Consumption Framework (S2C2F)



TIDALIS



@niels.fennec.dev



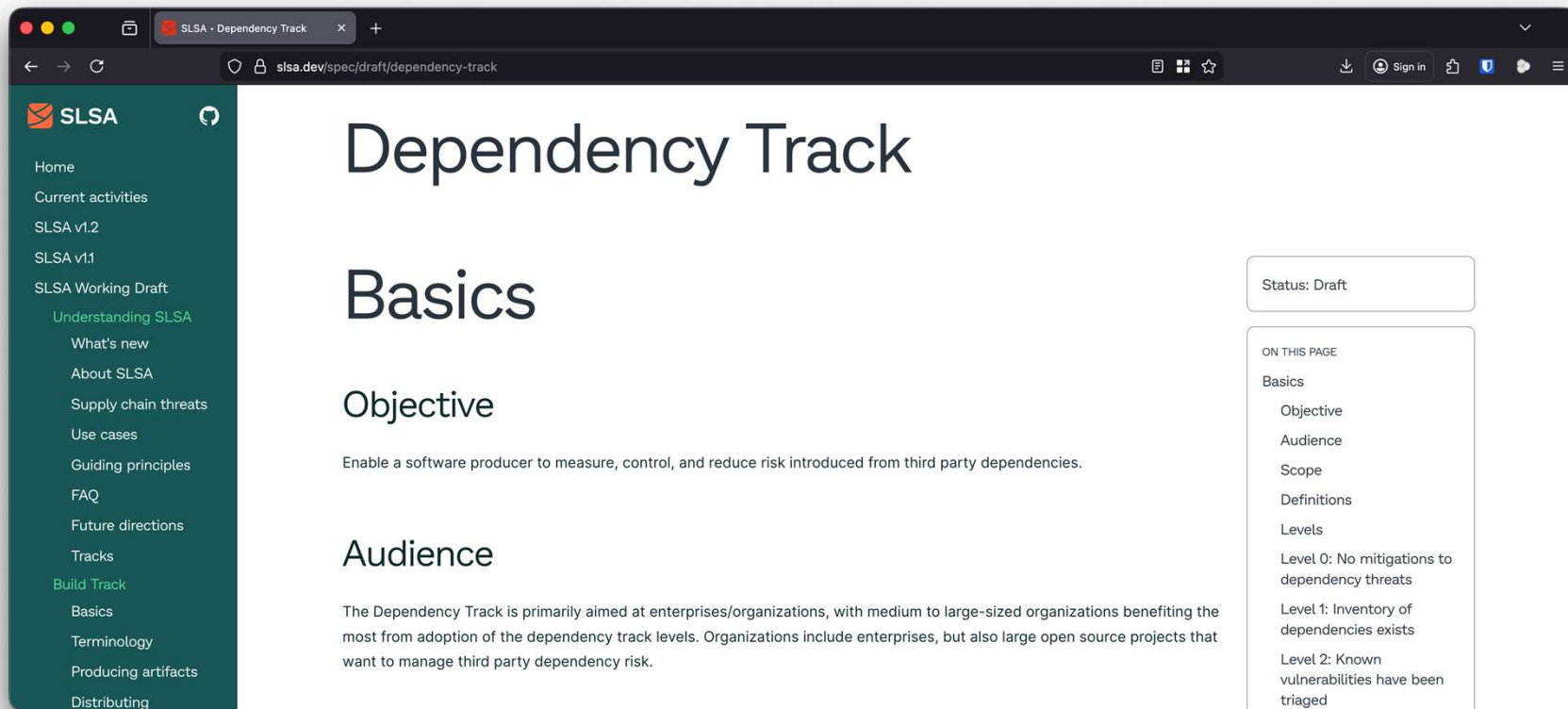
@nielstanis@infosec.exchange

Secure Supply Chain Consumption Framework (S2C2F)



Level 1	Level 2	Level 3	Level 4
 Minimum OSS Governance Program • Use package managers [ING-1] • Local copy of artifact [ING-2] • Scan with known vulns [SCA-1] • Scan for software licenses [SCA-2] • Inventory OSS [INV-1] • Manual OSS updates [UPD-1]	 Secure Consumption and Improved MTTR • Scan for end of life [SCA-3] • Have an incident response plan [INV-2] • Auto OSS updates [UPD-2] • Alerts on vulns at PR time [UPD-3] • Audit that consumption is through approved ingestion method [AUD-2] • Validate integrity of OSS [AUD-3] • Secure package source file configuration [ENF-1]	 Malware Defense and Zero-Day Detection • Deny list capability [ING-3] • Clone OSS source [ING-4] • Scan for malware [SCA-4] • Proactive security reviews [SCA-5] • Enforce OSS provenance [AUD-1] • Enforce consumption from curated feed [ENF-2]	 Advanced Threat Defense • Validate the SBOMs of OSS consumed [AUD-4] • Rebuild OSS on trusted infrastructure [REB-1] • Digitally sign rebuilt OSS [REB-2] • Generate SBOM for rebuilt OSS [REB-3] • Digitally sign protected SBOMs [REB-4] • Implement fixes [FIX-1]

S2C2F → SLSA Dependency Track



TIDALIS

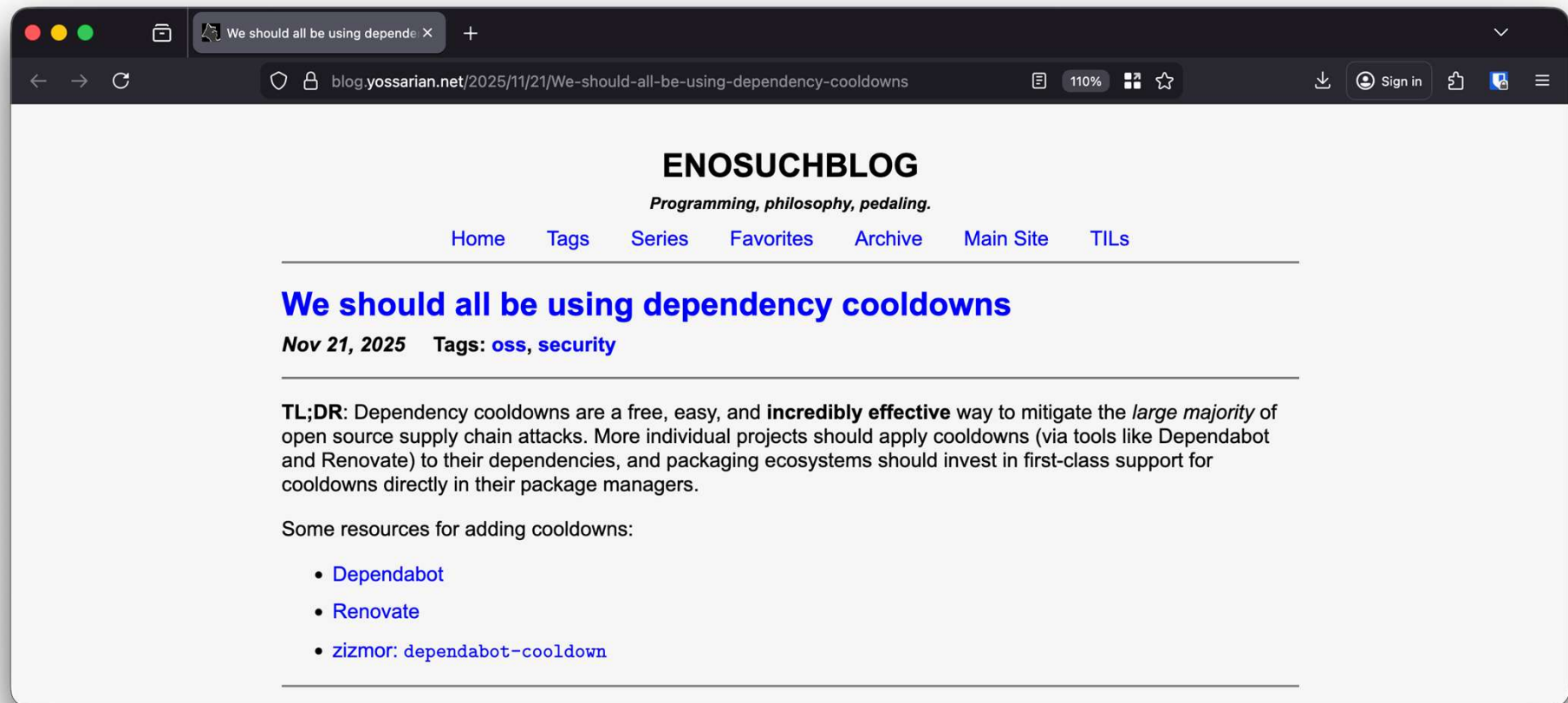


@niels.fennec.dev



@nielstanis@infosec.exchange

Hot take; dependency cooldowns



What's inside? (Audit)



TIDALIS



@niels.fennec.dev



@nielstanis@infosec.exchange

Open SSF Security Scorecard



- Set Automated Security Checks
- Scoring System 0-10
- Security Best Practices
- Ease of Use
- Community Support



TIDALIS



@niels.fennec.dev



@nielstanis@infosec.exchange

Maintenance



- Vulnerabilities (**High**)
 - Does the project have unfixed vulnerabilities?
- Maintained (**High**)
 - Is the project at least 90 days old, and maintained?
- Security Policy (**Medium**)
 - Does project contain security policy?

Maintenance



- License (**Low**)
 - Does the project declare a licence?
- CII Best Practices (**Low**)
 - Does the project have a CII Best Practices Badge?
- Dependency Update Tool (**High**)
 - Does the project use tools to help update its dependencies e.g. Dependabot, RenovateBot?

Continuous Testing



- Continuous Integration Tests (**Low**)
 - Does the project run tests in CI?
- Fuzzing (**Medium**)
 - Does the project use fuzzing tools, e.g. OSS-Fuzz?
- Static Analysis (**Medium**)
 - Does the project use static code analysis tools?

Source Risk Assessment



- Binary Artifacts (**High**)
 - Does repository contain binaries/compiled artifacts?
- Branch Protection (**High**)
 - GitHub best practices branch protection
- Dangerous Workflow (**Critical**)
 - GitHub best practices for Workflow Actions

Source Risk Assessment



- Code Review (**High**)
 - Does the project require code review before code is merged?
- Contributors (**Low**)
 - Does the project have contributors from multiple organizations?

Build Risk Assessment



- Pinned Dependencies (**Medium**)
 - Does the project declare and pin dependencies?
- Token Permission (**High**)
 - Does the project declare GitHub workflow tokens as read only?

Build Risk Assessment



- Packaging (**Medium**)
 - Does the project build and publish official packages from CI/CD?
- Signed Releases (**High**)
 - Does the project cryptographically sign releases?

Demo OpenSSF Scorecard



Running checks

TIDALIS



@niels.fennec.dev



@nielstanis@infosec.exchange

Fennec Labs Scorecard



WORK IN PROGRESS



TIDALIS



@niels.fennec.dev



@nielstanis@infosec.exchange

Fennec Labs Instrument



WORK IN PROGRESS



TIDALIS



@niels.fennec.dev



@nielstanis@infosec.exchange

Application Inspector

A screenshot of a web browser displaying the 'Application Inspector' interface. The browser's address bar shows a local file path. The page has a navigation bar with 'Overview', 'Summary', 'Features', and 'About' tabs. The main content area is titled 'Application Features' and includes a descriptive paragraph. Below this, there are two columns: 'Feature Groups' and 'Associated Rules'. The 'Feature Groups' column lists various categories like 'Select Features', 'General Features', 'Development', etc., each with a set of icons. The 'Associated Rules' column lists specific rules like 'Authentication: Microsoft (Identity)', 'Authentication: General', and 'Authentication: (OAuth)'.

Microsoft Application Inspector

file:///C:/Demo/AppInspector/publish/output.html#

Application Inspector Overview Summary Features About

Application Features

This section reports the major characteristics of the application or its primary features organized by customizable Feature Groups. Click any of the highlighted icons below (indicating at least one match) to view additional details or expand a feature group for more information. To view where in source code a specific feature was found, click the Rule name link shown on the right. A disabled icon indicates a not found status for that feature.

Feature Groups	Associated Rules
+ Select Features     	Name (click to view source)
+ General Features      	Authentication: Microsoft (Identity)
+ Development      	Authentication: General
+ Active Content      	Authentication: (OAuth)
+ Data Storage      	
+ Sensitive Data      	
+ Cloud Services        	
+ OS Integration        	
+ OS System Changes        	
+ Other       	



Application Inspector



Select Features

Feature	Confidence	Details
 Authentication		View
 Authorization		View
 Cryptography		View
 Object Deserialization		N/A
 AV Media Parsing		N/A
 Dynamic Command Execution		N/A

.NET Reproducibility



- Reproducible builds
 - Independent path from source to binary code.
- Roslyn Compiler Deterministic Compilation
- How reproducible is a simple console app?
 - Demo script available in repository



Fennec Labs Reproduce



WORK IN PROGRESS



TIDALIS



@niels.fennec.dev



@nielstanis@infosec.exchange

Fennec Labs Compare



WORK IN PROGRESS



TIDALIS



@niels.fennec.dev



@nielstanis@infosec.exchange

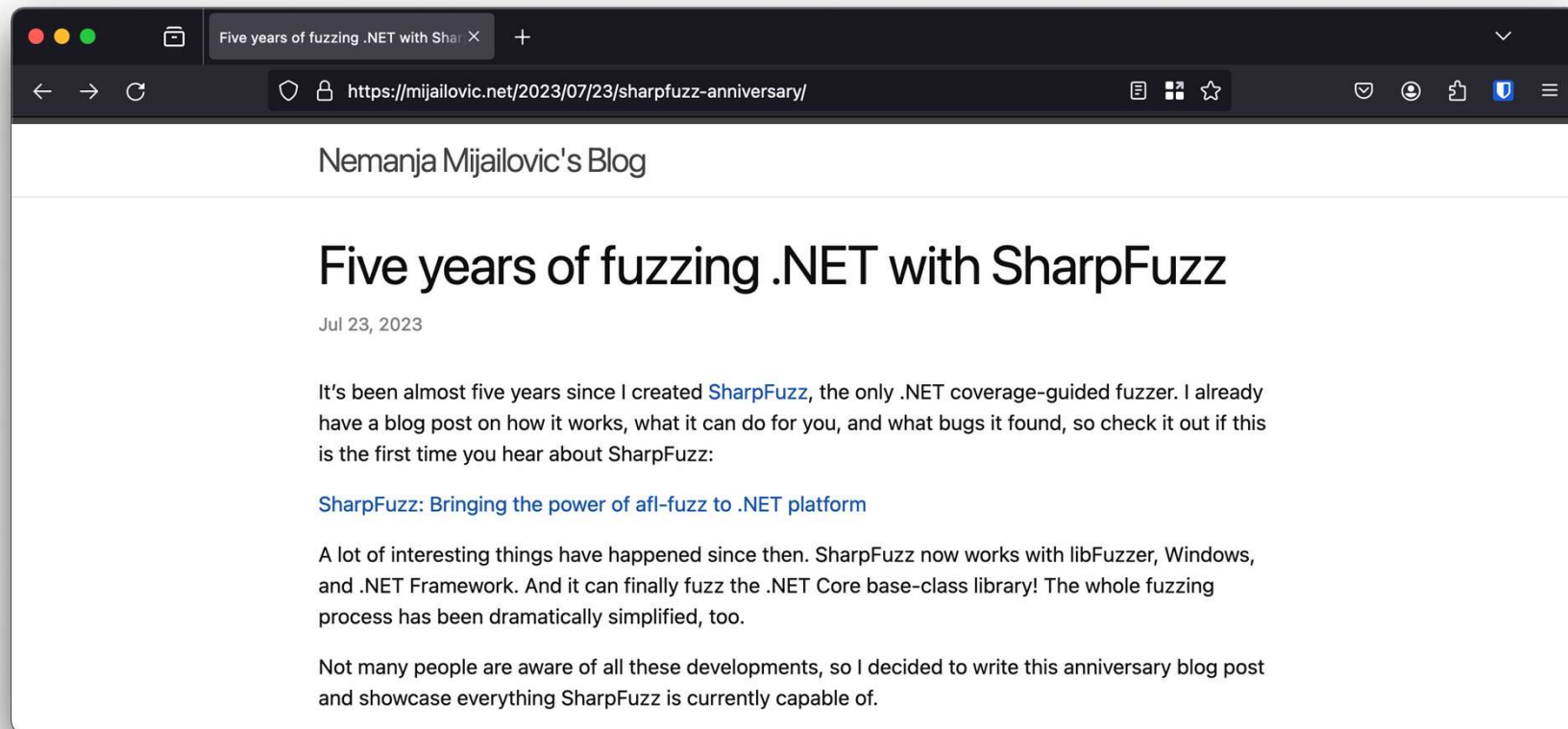
Future Idea - Fuzzing



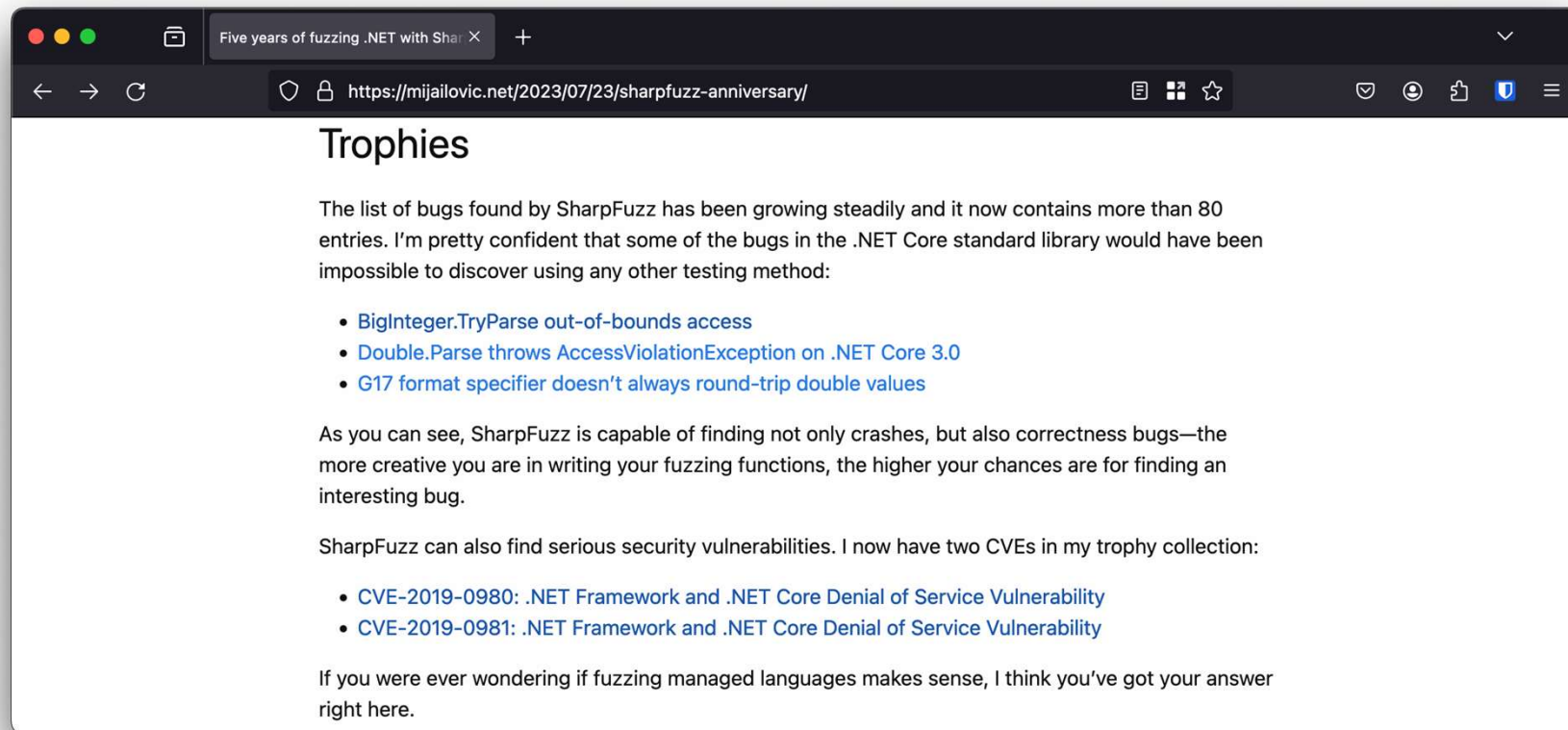
- Fuzzing, or fuzz testing
 - Automated software testing method that uses a wide range of **invalid** and unexpected data as input to find flaws
- Good in C/C++ memory issues
- Can it be of any value with managed languages like .NET?



Fuzzing .NET & SharpFuzz



Fuzzing .NET & SharpFuzz



Fuzzing .NET – Jil JSON Serializer



```
public static void Main(string[] args)
{
    SharpFuzz.Fuzzer.OutOfProcess.Run(stream => {
        try
        {
            using (var reader = new
                System.IO.StreamReader(stream))
                JSON.DeserializeDynamic(reader);
        }
        catch (DeserializationException) { }
    });
}
```


Fuzzomatic: Using AI to Fuzz Rust

A screenshot of a web browser window displaying an article titled 'Introducing Fuzzomatic: Using AI to Fuzz Rust'. The browser's address bar shows the URL: https://research.kudelskisecurity.com/2023/12/07/introducing-fuzzomatic-using-ai-to-automatically-fuzz-rust-projects-from-scratch/. The article content includes a section 'How does it work?' explaining that Fuzzomatic uses libFuzzer and cargo-fuzz as a backend and leverages AI models like gpt-3.5-turbo. It also discusses 'Fuzz targets and coverage-guided fuzzing' and provides a Rust code snippet for a fuzz target. The code defines a 'fuzz_target!' macro that takes a byte slice and converts it to a UTF-8 string to call a target function. At the bottom of the article, there are social media interaction buttons for 'Comment', 'Reblog', 'Subscribe', and a menu icon.

Introducing Fuzzomatic: Using AI to Fuzz Rust

https://research.kudelskisecurity.com/2023/12/07/introducing-fuzzomatic-using-ai-to-automatically-fuzz-rust-projects-from-scratch/

How does it work?

Fuzzomatic relies on libFuzzer and cargo-fuzz as a backend. It also uses a variety of approaches that combine AI and deterministic techniques to achieve its goal.

We used the OpenAI API to generate and fix fuzz targets in our approaches. We mostly used the gpt-3.5-turbo and gpt-3.5-turbo-16k models. The latter is used as a fallback when our prompts are longer than what the former supports.

Fuzz targets and coverage-guided fuzzing

The output of the first step is a source code file: a fuzz target. A libFuzzer fuzz target in Rust looks like this:

```
1  #![no_main]
2
3  extern crate libfuzzer_sys;
4
5  use mylib_under_test::MyModule;
6  use libfuzzer_sys::fuzz_target;
7
8  fuzz_target!(|data: &[u8]| {
9      // fuzzed code goes here
10     if let Ok(input) = std::str::from_utf8(data) {
11         MyModule::target_function(input);
12     }
13 });
```

This fuzz target needs to be compiled into an executable. As you can see, this program depends on libFuzzer and also depends on the library under test, here "mylib_under_test". The "fuzz_target!" macro makes it easy for us to just write what needs to be called, provided that we receive a byte slice, the "data" variable in the above example. Here we convert these bytes to a UTF-8 string and call our target function and pass that string as an argument. LibFuzzer takes care of calling our fuzz target repeatedly with random bytes. It measures the code coverage to assess whether the random input helps cover more code. We say it's coverage-guided fuzzing.

Comment Reblog Subscribe ...

Future Idea – Dashboard & Feedback



- Local dashboard for project
- Remote dashboard for community
- Ability to do review and share security related data of project
- MCP server with embedded prompt resources to help security review
- Roslyn analyzer that can help guide the developer



Conclusion



- Doing software security is hard and Software Security Professionals don't scale!
- Package eco-systems have been wild west lately
- Goal is to have Fennec Labs help with reviewing and sharing security details for all audiences
- Allowing everyone to make better informed decisions and can act if needed.



Thank you!



- niels at fennec.dev
- <https://github.com/nielstanis/futuretech2026>
- Questions?



TIDALIS



@niels.fennec.dev



@nielstanis@infosec.exchange